

## Lab 3: Sorting Performance and Stability

**Due: 11:59pm** on April 01, 2025 (Tuesday)

In Module 8, we saw three different elementary sorting algorithms: Bubble Sort, Selection Sort, and Insertion Sort.

Even though their worst-case running time complexity in Big O Notation is the same,  $O(n^2)$ , we saw that there can be a difference in their real performance.

It's time to see it for yourself.

First, download `SimpleSortDemo.java` and `Stopwatch.java` files from Canvas and run them on your machine.

### Step 1: Let's quickly look at their running times.

- What is the running time of bubble sort?
- What is the running time of selection sort?
- What is the running time of insertion sort?

Try to run the driver program with a different array length to verify the trends you see.

### Step 2: Let's change the order of the values in the arrays

Comment out Case 1 (for random input values) and uncomment Case 2 (for ordered input values) and run the program again.

What do you notice?

### Now, it is time to refresh your memory!

1) Can you find one change you can make in the bubble sort method to make it a little faster? Try to make that change and run the driver again.

2) Why can selection sort generally be faster than bubble sort?

3) Why can insertion sort be faster than selection sort and bubble sort?

4) Under what circumstances can insertion sort get close to  $O(n)$  running time complexity (where  $n$  is the number of elements in an array)?

## Sorting and Stability

Sometimes, we care about what happens to data items with equal keys during sorting.

For example, you have employee data arranged alphabetically by last names. You want to sort them again by ZIP code but keep the original order among the employees with the same last name as it used to be.

Some sorting algorithms retain this secondary ordering. We say those sorting algorithms are *stable*.

### Problem:

Download the other three lab files (SelectionSortApp.java, InsertionSortApp.java, and Employee.java) from Canvas for this part. Given the Employee class (also shown below), write code to sort (one using insertion sort and the other using selection sort, in their respective Java files) by last name or by zip code in ascending order.

```
public class Employee {
    private String firstName;
    private String lastName;
    private int zipCode;
    public Employee(String fName, String lName, int zCode) {
        firstName = fName;
        lastName = lName;
        zipCode = zCode;
    }
    public String getLastName() {
        return lastName;
    }
    public int getZipCode() {
        return zipCode;
    }
    @Override
    public String toString() {
        return lastName + ", " + firstName + ", " + zipCode;
    }
}
```

Once you are done, run your code.

1. What is your output for insertion sort by last name?
2. What is your output for selection sort by last name?
3. How about insertion sort by zip code that tries to sort the Employee list that was already sorted by last name? Does it keep the secondary order?
4. How about selection sort by zip code that tries to sort the Employee list that was already sorted by last name? Does it keep the secondary order?
5. Do you see any difference? If so, can you explain the reason for the difference?

**<Hint> We know that  $B = b$  and  $a < b < c$  and you have  $\{B, b, a, c\}$  to sort. The result after the first iteration of selection sort is different from the result after the first iteration of insertion sort.**

Place InsertionSortApp.java and SelectionSortApp.java files into a zip file called lab3.zip.

Submit your lab3.zip file using Autolab (<https://autolab.andrew.cmu.edu>). Make sure to zip .java files, not .class files.

The questions in this handout are for you to think about. You are not required to submit this handout.